

OS Questions

1. What is the difference between a process and a thread?

- A process is an independent program in execution with its own dedicated memory space (including code, heap, and stack), registers, and system resources.
- A thread is the smallest unit of execution within a process.
- Processes are completely isolated from one another, whereas multiple threads can exist inside a single process and communicate easily by sharing the same memory space.

2. What is context switching and why is it expensive?

- Context switching is the act of saving the state of a currently running process or thread so that it can be paused, and loading the state of another process to run.
- It is computationally expensive because the operating system must perform multiple heavy operations, such as saving CPU register values, saving the program counter, flushing and reloading TLB/cache entries, and updating scheduling data structures.

3. What is Virtual Memory?

- Virtual memory is a memory management technique that allows programs larger than the available physical RAM to execute.
- It provides process isolation, simplifies overall memory management, and enables multitasking.
- The operating system utilizes and maintains page tables to translate these virtual addresses into physical addresses in the hardware.

4. What is Demand Paging?

- Demand paging is a strategy where a memory page is loaded into RAM only when it is explicitly accessed, rather than loading it beforehand.
- When a program tries to access a page that is not currently in memory, a "page fault" occurs.

- The operating system then steps in, loads the required page from the disk into the RAM, updates the page table, and resumes execution.

5. What is Thrashing?

- Thrashing is a severe performance degradation that occurs when a system spends more time paging (swapping memory pages in and out of the disk) than executing actual program instructions.
- It can be mitigated by reducing the degree of multiprogramming, implementing a working set model, or improving the system's paging algorithms.

6. What is a Deadlock and what are its necessary conditions?

- A deadlock is a state where a set of processes are permanently blocked because each process is holding a resource while waiting for another resource that is currently acquired by a different process.
- Four necessary conditions must hold simultaneously for a deadlock to occur: mutual exclusion, hold and wait, no pre-emption, and circular wait.

7. What is a Semaphore?

- A semaphore is a synchronization mechanism used to prevent race conditions and ensure mutual exclusion when accessing shared resources.
- A **Binary Semaphore** operates strictly with values of {0,1} and functions similarly to a mutex lock.
- A **Counting Semaphore** allows multiple threads to access multiple available instances of a specific resource.

8. What is the difference between Preemptive and Non-Preemptive scheduling?

- In **Preemptive scheduling**, the CPU is allocated to a process for a limited amount of time. An executing process can be interrupted and paused if a higher-priority process arrives in the queue. This method carries a higher overhead due to frequent context switching.
- In **Non-Preemptive scheduling**, a running process cannot be interrupted; it retains control of the CPU until it finishes its task or voluntarily yields.

9. What is the difference between Logical and Physical Address Space?

- A **logical address** is generated directly by the CPU and is used by the user's program to refer to memory locations.
- A **physical address** is computed by the hardware's Memory Management Unit (MMU).
- Users and programs interact with logical addresses but can only indirectly access the actual physical addresses in the RAM.

10. What is a Kernel and how does it differ from the Operating System?

- The **kernel** is the core, most essential component of an operating system. It operates in a privileged mode and is strictly responsible for managing system resources, the processor, memory, and device drivers.
- While the **operating system** as a whole provides a broad environment for process management, disk management, and user interfaces, the kernel is the specific low-level software that facilitates direct communication between the hardware and those higher-level software components.